¿Como crear un proyecto con 'uv'?

uv es una herramienta moderna, unificada y muy rápida par ala gestión de proyectos, paquetes y entornos de **Python**, desarrollada en **Rust** por la empresa Astral.

Esta diseñada para reemplazar y acelerar tareas que antes requerían múltiples herramientas tradicionales como "*pip*", "*virtualenv*", "*poetry*" o "*pyenv*".

Sus principales caracteristicas consisten en uina gestión transparente automatizando entornos virtuales y la instalación de librerías con comandos sensillos "*uv add*", "*uv run*", "*uv sync*", y la reproducción total utilizando el bloqueo "*uv.lock*" para asegurar que el proyecto funcione con las versiones exactas de las dependanceias en cualquier computadora.

A continuación, se detalla paso a paso como se crea un proyecto desde cero.

Creando un proyecto

En terminal escribimos:

```
mkdir pre_proces_metereo
cd pre_proces_metereo
uv init
```

¿Qué significa cada línea?

```
mkdir pre_proces_metereo
```

Esta línea viene de *Make Directory* y crea un directorio. con esto crea una carpeta vacía en la computadora que servirá como contenedor principal de todo el proyecto.

```
cd pre_proces_metereo
```

Esta línea viene de *Change Directory* y le indica a la terminal a que carpeta se debe de dirigir, para que todas las acciones ocurran dentro de este espacio.

```
uv init
```

Con esta línea se inicia *uv*, transformando la carpeta común en un proyecto de *Python* estructurado. Además crea automáticamente archivos clave como *pyproject.toml* (configuración)

y *uv.lock* (control de versiones exactas), ademas de un entorno virtual oculto.

Añadiendo dependencias al proyecto

Para que el proyecto funcione en optimas condiciones, necesitamos instalar librerias externas (en este caso se instalarán *pandas* para el tratamiento de datos y *numpy* para operaciones numéricas). la unica diferencia es que cambiamos herrmaientas tradicionales como *pip*, y ejecutamos:

```
uv add pandas numpy
```

este comando descarga e instala de manera rapida las librerias especificadas, vinculándolas automáticamente al entorno virtual del proyecto y actualizando los archivos de control de versiones (*pyproject.toml* y *uv.lock*) para asegurar que cualquier persona pueda clonar el repositorio con las mismas verisones

Ejecución del proyecto

Una vez escrito tu código, no necesitas activar manualmente ningún entorno virtual. Solo se utiliza:

```
uv run main.py
```

Este comando ejecuta el archivo principal (*main.py*) utilizando de forma transparente el entorno virtual del proyecto, garantizando que todas las librerías necesarias estén disponibles de inmediato.